

Cahier des Charges

Gestionnaire Métier — Interface Graphique

1. CONTEXTE ET OBJECTIFS

Vous allez concevoir et développer en groupe de 3 un gestionnaire métier complet avec interface graphique. L'objectif est de mettre en pratique les concepts avancés de la programmation orientée objet à travers une application concrète et fonctionnelle.

Même sujet, rendus différents : les choix de conception, les fonctionnalités bonus et l'interface seront propres à chaque groupe.

2. ARCHITECTURE IMPOSEE

L'architecture doit respecter les contraintes OO suivantes. Ces contraintes sont évaluées à la soutenance.

◆ 2.1 Classes abstraites

Vous devez définir au minimum 2 classes abstraites représentant les concepts centraux de votre domaine.

Domaine	Exemples de classes abstraites
Hôpital	Personne, Personnel, Soin
Flotte de véhicules	Véhicule, Mission, Incident

◆ 2.2 Interfaces multiples

Au moins 3 interfaces métier distinctes doivent être définies et implémentées. Une même classe peut en implémenter plusieurs.

Domaine	Exemples d'interfaces
Hôpital	Soignable, Planifiable, Urgence, Facturable
Flotte de véhicules	Assignable, Maintenable, Trackable, Facturable

◆ 2.3 Generics

Vous devez utiliser au moins un type générique significatif dans votre modèle. Le type doit être borné (extends).

- Exemple : `Registre<T extends Entite>` pour gérer n'importe quelle collection métier
- Exemple : `File<T extends Urgence>` pour prioriser des entités urgentes

- Utilisation de wildcards (? extends, ? super) appréciée

◆ 2.4 Collections avancées

Les structures de données doivent être choisies de façon justifiée. Les types bruts (raw types) sont interdits.

Collection	Usage attendu
List<T>	Listes ordonnées (historique, planning)
Map<K,V>	Associations clé-valeur (index, agenda)
Set<T>	Unicité (identifiants, droits)
PriorityQueue<T>	File de priorité (urgences, maintenance)
TreeMap / TreeSet	Collections triées naturellement

◆ 2.5 Lambdas et Streams

Les traitements sur les collections doivent utiliser l'API Stream Java 8+. Les boucles for classiques sont tolérées mais les streams sont attendus pour le filtrage, le tri et l'agrégation.

- filter() pour la recherche multicritères
- sorted() avec Comparator pour les tris
- map() / collect() pour les transformations
- Lambdas pour les Comparator et Predicate passés en paramètre

3. FONCTIONNALITES ATTENDUES

Chaque application doit implémenter un noyau fonctionnel commun, complété par des fonctionnalités propres au domaine choisi.

◆ 3.1 Fonctionnalités communes (obligatoires)

Fonctionnalité	Description
CRUD complet	Créer, lire, modifier, supprimer toute entité principale
Filtrage multicritères	Recherche par au moins 3 critères combinables
Tri dynamique	Tri sur au moins 2 colonnes, ordre croissant/décroissant
Statistiques	Au moins 3 indicateurs calculés dynamiquement
Persistance légère	Sauvegarde/chargement via sérialisation Java ou CSV
Exceptions métier	Exceptions custom (ex : CapacitéDépasséeException)

◆ 3.2 Fonctionnalités spécifiques — Hôpital

- Gestion des patients : admission, sortie, antécédents, dossier médical

- Gestion du personnel : médecins, infirmiers, spécialités, plannings
- Gestion des soins : consultations, prescriptions, actes médicaux
- Gestion des salles et lits : occupation, type (urgences, chirurgie...)
- File d'attente aux urgences avec priorité médicale

Exemple de hiérarchie — Hôpital

Personne (abstraite)

- └─ Personnel (abstraite)
 - └─ Médecin implements Soignable, Planifiable
 - └─ Infirmier implements Planifiable
- └─ Patient implements Soignable, Facturable

Soin (abstraite)

- └─ Consultation
- └─ ActeChirurgical implements Urgence

◆ 3.3 Fonctionnalités spécifiques — Flotte de véhicules

- Gestion des véhicules : immatriculation, type, état, kilométrage
- Gestion des chauffeurs : permis, disponibilité, historique de missions
- Gestion des missions : affectation, itinéraire, statut, rapport de fin
- Suivi de la maintenance : entretiens planifiés, incidents, coûts
- Tableau de disponibilité temps réel (véhicules / chauffeurs)

Exemple de hiérarchie — Flotte

Véhicule (abstraite)

- └─ VéhiculeLéger implements Assignable, Maintenable
- └─ VéhiculeLourd implements Assignable, Maintenable
- └─ VéhiculeSpécial implements Urgence, Maintenable

Mission (abstraite)

- └─ MissionCourte
- └─ MissionLongue implements Trackable

4. INTERFACE UTILISATEUR

Chaque groupe choisit librement sa technologie d'interface parmi deux options. La logique métier (modèle, collections, generics, streams) reste identique quel que soit le choix.

Option A — Swing	Option B — Web (Servlet/JSP)
Application desktop Java <i>JFrame, JPanel, JTable, JDialog</i> TableModel personnalisé SwingUtilities.invokeLater() Pas de logique métier dans les listeners	Application web Java EE légère <i>Servlets + JSP (Tomcat)</i> Séparation Servlet (contrôleur) / JSP (vue) Passage de données via request.setAttribute() HTML/CSS libre (Bootstrap accepté)

◆ 4.1 Exigences communes aux deux options

- Au moins 3 vues/pages distinctes (liste, détail, statistiques)
- Affichage tabulaire des collections avec tri et filtrage
- Formulaires de saisie avec validation des données
- Feedback utilisateur : messages d'erreur et confirmations
- Mise à jour dynamique de l'affichage après chaque modification

◆ 4.2 Exigences spécifiques — Option A (Swing)

- JTable avec TableModel personnalisé, tri sur clic d'en-tête
- SwingUtilities.invokeLater() pour toutes les mises à jour UI
- Aucun code métier dans les ActionListener — déléguer au modèle

◆ 4.3 Exigences spécifiques — Option B (Servlet/JSP)

- Un Servlet par entité principale (ex : PatientServlet, MedecinServlet)
- Les JSP n'accèdent au modèle que via les attributs de requête/session
- Le modèle métier (classes Java) est totalement indépendant de la couche web
- Déploiement sur Tomcat 10+ démontré en soutenance

◆ 4.4 Ce qui différenciera les groupes

- Graphiques et visualisations (JFreeChart côté Swing, Chart.js côté web)
- Thème graphique soigné (couleurs, icônes, mise en page)
- Recherche en temps réel, raccourcis clavier (Swing) ou AJAX (web)
- Export CSV ou impression des données
- Calendrier visuel, planning ou carte de disponibilités

5. RENDU — DEPOT GIT

Le rendu se fait exclusivement via un dépôt Git (GitHub, GitLab ou Bitbucket).

◆ 5.1 Structure attendue du dépôt

Structure du projet

```
mon-projet/
├── src/
│   ├── model/      ← Classes métier, abstraites, interfaces
│   ├── view/       ← Composants Swing ou Servlets/JSP
│   ├── controller/ ← Logique applicative
│   └── util/        ← Génériques, helpers
├── resources/      ← Données de test (.csv, .ser...)
├── README.md
└── rapport_conception.pdf
```

◆ 5.2 README.md obligatoire

- Nom du projet et du groupe (prénoms + noms)
- Domaine métier et description en 5 lignes
- Technologie d'interface choisie (Swing ou Servlet/JSP) et justification
- Instructions de compilation et d'exécution (ou de déploiement Tomcat)
- Liste des fonctionnalités implémentées
- Répartition des tâches entre les membres

◆ 5.3 Rapport de conception (PDF, 3 à 5 pages)

- Diagramme de classes UML (au moins les classes principales)
- Justification des choix OO (pourquoi abstraite ici, interface là ?)
- Difficultés rencontrées et solutions adoptées
- Ce qui aurait été amélioré avec plus de temps

◆ 5.4 Qualité des commits

L'historique Git sera observé. Chaque membre doit avoir des commits significatifs.

- Messages de commit clairs, en français ou en anglais
- Commits réguliers — pas un seul commit final

- Chaque membre visible dans l'historique

6. SOUTENANCE

La soutenance dure 20 minutes par groupe, répartie comme suit :

Phase	Durée
Démo live de l'application	8 min
Présentation des choix de conception OO	7 min
Questions de l'enseignant	5 min

Chaque membre doit être en mesure d'expliquer n'importe quelle partie du code, y compris celle écrite par ses coéquipiers.

7. GRILLE D'EVALUATION

Critère	Points
Classes abstraites (présence, pertinence, héritage cohérent)	4 pts
Interfaces multiples (définition, usage, polymorphisme)	4 pts
Generics bornés (pertinence, wildcards)	3 pts
Collections & Streams / Lambdas	3 pts
Fonctionnalités métier complètes	4 pts
Interface (ergonomie, réactivité, qualité — Swing ou Web)	4 pts
Qualité du code (structure, nommage, exceptions)	3 pts
Dépôt Git (commits, README, rapport)	3 pts
Soutenance (clarté, maîtrise, démo)	4 pts
Fonctionnalités bonus et originalité	+2 pts bonus
TOTAL	32 pts + 2 bonus

Rappel important

Deux groupes sur le même domaine auront des similitudes dans le modèle métier.
La différenciation se fait sur : les fonctionnalités choisies, la qualité de l'interface,
la richesse des statistiques, la profondeur du modèle OO et l'originalité des choix.

Un groupe qui réutilise le code d'un autre sera sanctionné.